

Adam Optimizer Explained

Optimizers in Deep Learning — Part 5 | Notes based on the CampusX video

1. What is Adam?

Adam stands for **Adaptive Moment Estimation**. It is, honestly, the most widely used and currently the most powerful optimization technique in common use. Most of the time, when training an ANN, CNN, or RNN, the optimizer of choice is Adam.

Adam builds directly on the ideas from the optimizers covered earlier in this series: Batch / Mini-batch Gradient Descent (the most basic technique), Momentum, NAG (Nesterov Accelerated Gradient), AdaGrad, and RMSProp. Adam borrows concepts from these earlier techniques, so understanding it is much easier once you're familiar with that progression.

2. Quick Recap: The Story So Far

Before diving into Adam's formula, it helps to recall, briefly, what each prior optimizer contributed and what problem it ran into:

- **Batch Gradient Descent:** Quietly descends into a valley toward the minimum, but its only problem is speed — it simply doesn't move fast enough.
- **Momentum:** Solves the speed problem by using past gradients (velocity) for the current update. Where Batch Gradient Descent might take 10–12 steps to descend a valley, Momentum can do it in 2–3. But this introduces a new problem: **oscillations** — overshooting, coming back, overshooting again, slowly settling near the minimum.
- **NAG (Nesterov Accelerated Gradient):** Dampens Momentum's oscillations by looking ahead before updating, producing a smaller, more controlled path (a smaller "circle" of oscillation instead of a larger one).
- **AdaGrad:** Solves a different problem — sparse features. With Momentum or NAG, if a column is sparse, the optimizer first descends quickly along the dense (non-sparse) direction and only slowly shifts toward the sparse direction, which is inefficient. AdaGrad fixes this by decaying the learning rate per parameter, descending directly even with sparse data.
- **RMSProp:** AdaGrad's own weakness was that it decays the learning rate so aggressively that updates can stop almost entirely before reaching the minimum, causing it to slightly miss the target. RMSProp fixes this and descends smoothly and precisely all the way to the minimum.

Looking at this whole journey, you can see it really revolves around **two core ideas**:

- **The idea of Momentum** — used by Momentum and NAG.
- **The idea of adaptive learning rate decay** — used by AdaGrad and RMSProp.

The natural next question: what if you combined both ideas into a single optimizer? That combination is exactly the core principle behind Adam.

3. Mathematical Formulation of Adam

Adam maintains two separate running estimates for each parameter:

$$\mathbf{m}_t = \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \nabla L(\mathbf{w}_t)$$
$$\mathbf{v}_t = \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot [\nabla L(\mathbf{w}_t)]^2$$

Here, $\mathbf{m}_t$ is the Momentum-style term (the "first moment" — an exponentially weighted average of past gradients), and $\mathbf{v}_t$ is the RMSProp-style term (the "second moment" — an exponentially weighted average of past squared gradients, used to adapt the learning rate). These are exactly the two core ideas recapped above, now combined into one optimizer.

Before using them in the final update, both terms go through a **bias correction** step:

$$\mathbf{m_hat}_t = \mathbf{m}_t / (1 - \beta_1^t) \quad | \quad \mathbf{v_hat}_t = \mathbf{v}_t / (1 - \beta_2^t)$$

Bias correction is needed because at the very start of training, both $\mathbf{m}$ and $\mathbf{v}$ are initialized to zero ($\mathbf{m}_0 = 0$ and $\mathbf{v}_0 = 0$). This creates a bias toward zero in the early steps. Dividing by $(1 - \beta^t)$ offsets this early-step bias. This is purely a mathematical detail; the underlying core idea matters far more than the bias-correction mechanics themselves — in practice you can rely on existing library implementations for this part.

Finally, the weight update rule is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - [\alpha / (\sqrt{\mathbf{v_hat}_t} + \epsilon)] \cdot \mathbf{m_hat}_t$$

Symbol	Meaning
$\mathbf{m}_t$	First moment estimate — the Momentum-style exponentially weighted average of gradients
$\mathbf{v}_t$	Second moment estimate — the RMSProp-style exponentially weighted average of squared gradients
β_1	Decay rate for the first moment, typically 0.9
β_2	Decay rate for the second moment, typically 0.999
α (alpha)	The base (global) learning rate
ϵ (epsilon)	A very small constant added to avoid division by zero

Typical default values: $\beta_1 = 0.9$ and $\beta_2 = 0.999$. These are configurable hyperparameters, but Adam's main practical advantage is that it rarely needs much learning-rate tuning at all — it largely handles that automatically.

The computation order at each step is: first compute $\mathbf{m}_t$, then compute $\mathbf{v}_t$, then apply bias correction to get $\mathbf{m_hat}_t$ and $\mathbf{v_hat}_t$, and finally plug these into the weight update formula.

4. How Adam Performs: What the Animation Shows

Using a dataset with a sparse column, the video animates Adam alongside the earlier optimizers. The animation illustrates both behaviors Adam inherits at once: the Momentum-style behavior and the adaptive learning-rate-decay behavior.

The original (plain) Momentum approached the minimum from one particular direction. Adam, by contrast, approaches in a more centered, optimized way and settles into the minimum region directly — combining the fast convergence of Momentum-style optimizers with the precision of adaptive learning-rate optimizers. On this example, Adam also converges in fewer steps than the other optimizers shown.

Note: on this particular (relatively simple, convex) example, the full advantage of Adam isn't fully obvious. Its real strength shows up on **complex, non-convex optimization problems** — like those found in real neural networks — where Adam tends to perform very well.

5. The Verdict: Which Optimizer Should You Use?

Beyond the optimizers covered in this series, there are others too (such as AdaDelta) that weren't discussed here. So which one should you actually use?

In most cases, if you're working in deep learning, **Adam is a good starting point**. Try running Adam first. If the results aren't satisfactory, you can try RMSProp instead. Occasionally, plain Momentum even outperforms the others on certain problems. In short, there's no single universally "correct" answer.

That said, over the last several years, Adam has generally delivered better results across a wide variety of problem types compared to the alternatives. So a reasonable practical strategy is: **start with Adam**, and if you're not satisfied with the results, treat the optimizer choice as a hyperparameter and experiment to see what works best for your specific data and problem.

6. Summary

Adam (Adaptive Moment Estimation) combines the two major ideas developed across this optimizer series: the **Momentum** idea (an exponentially weighted average of past gradients, m_t) and the **adaptive learning rate** idea from RMSProp (an exponentially weighted average of past squared gradients, v_t). After applying bias correction to offset their zero initialization, these two terms are combined into a single weight-update rule. This combination gives Adam both fast, momentum-driven convergence and the per-parameter learning-rate adaptiveness needed for sparse or differently-scaled features, with comparatively little manual learning-rate tuning required. It is widely regarded as a strong default choice for training neural networks, though it is not guaranteed to be optimal for every problem, and alternatives like RMSProp or Momentum are worth trying if results aren't satisfactory.

Reference: Video Timestamps

Source video: "Adam Optimizer Explained in Detail with Animations | Optimizers in Deep Learning Part 5"
(CampusX)

00:00 — Intro

00:40 — What is Adaptive Moment Estimation? (ADAM)

05:45 — Math behind ADAM

11:24 — The Verdict

Notes compiled for personal study based on the CampusX YouTube lecture on the Adam Optimizer. For the full explanation with animations, refer to the original video.